



UNIVERSIDAD PRIVADA DEL VALLE

FACULTAD: FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
CARRERA: INGENIERÍA DE SISTEMAS INFORMÁTICOS
MATERIA: TECNOLOGÍAS EMERGENTES I

PROYECTO + EXPOSICIÓN FINAL

MATERIA: Tecnologías Emergentes I

DOCENTE: Ing. Efrain Luna Mamani

ESTUDIANTES: Nicolas Mateo Zamalloa Maldonado

Carla Andrea Camacho Sanizo

FECHA PRESENTACIÓN INFORME: 14/06/2026



1. Introducción

El fraude financiero representa uno de los principales desafíos para instituciones bancarias y plataformas de pago digitales. A medida que aumenta el volumen de transacciones electrónicas, también crecen las oportunidades para que actores maliciosos realicen actividades fraudulentas, generando pérdidas económicas significativas y afectando la confianza de los usuarios en los sistemas financieros.

El objetivo de este proyecto es desarrollar un modelo de aprendizaje automático capaz de identificar si una transacción es fraudulenta o legítima a partir de diferentes características relacionadas con el cliente, el método de pago y el contexto de la operación. Para ello, se emplean técnicas de clasificación supervisada que permiten aprender patrones presentes en datos históricos y realizar predicciones sobre nuevas transacciones.

La detección automática de fraudes tiene una gran relevancia en el ámbito financiero debido a que permite reducir pérdidas económicas, mejorar la seguridad de los usuarios y optimizar los sistemas de monitoreo. Además, el uso de modelos predictivos posibilita identificar actividades sospechosas en tiempo real, facilitando respuestas rápidas y eficientes ante posibles amenazas.

2. Descripción del Dataset

El conjunto de datos utilizado corresponde a registros de transacciones financieras, donde cada observación representa una operación realizada por un cliente. El objetivo es determinar si una transacción corresponde a un fraude mediante la variable objetivo **is_fraud**.

El dataset contiene variables relacionadas con:

- Información temporal de la transacción.
- Características del cliente.
- Información financiera.
- Ubicación geográfica.
- Método de pago y dispositivo utilizado.
- Indicadores de comportamiento sospechoso.

La variable objetivo es:

- **is_fraud**:
 - 0: Transacción legítima.
 - 1: Transacción fraudulenta.

Además, existe una variable adicional denominada **fraud_type**, la cual especifica el tipo de fraude cuando corresponde.



2.1. Variables del dataset

Variable	Descripción
transaction_id	Identificador único de la transacción
customer_id	Identificador del cliente
transaction_date	Fecha de la transacción
transaction_time	Hora de la transacción
hour_of_day	Hora del día en formato numérico
is_weekend	Indica si ocurrió en fin de semana
is_night_transaction	Indica si fue realizada durante la noche
country	País donde ocurrió la transacción
city	Ciudad asociada a la transacción
merchant_category	Categoría del comercio
payment_method	Método de pago utilizado
device_type	Tipo de dispositivo empleado
customer_age	Edad del cliente
credit_score	Puntaje crediticio del cliente
account_age_years	Antigüedad de la cuenta en años
account_balance	Saldo de la cuenta
transaction_amount	Monto de la transacción
num_prev_transactions	Número de transacciones previas
transaction_freq_monthly	Frecuencia mensual de transacciones
distance_from_home_km	Distancia desde el hogar del cliente
time_since_last_txn_hrs	Tiempo desde la última transacción
is_international	Indica si la transacción es internacional
failed_attempts	Número de intentos fallidos



pin_changed_recently	Indica si el PIN fue cambiado recientemente
is_fraud	Variable objetivo (fraude o no fraude)
fraud_type	Tipo de fraude detectado

2.2. Tamaño del dataset

El conjunto de datos utilizado está compuesto por aproximadamente **1 000 000 de registros** y **26 variables**, incluyendo la variable objetivo `is_fraud`. La gran cantidad de observaciones permite entrenar modelos robustos capaces de identificar patrones complejos asociados al fraude financiero.

2.3. Fuente del dataset

El dataset fue obtenido de la plataforma Kaggle, un repositorio ampliamente utilizado para proyectos de ciencia de datos y aprendizaje automático.

<https://www.kaggle.com/datasets/nafiulislam490/bank-transaction-fraud-detection-dataset>

2.4. Distribución de los datos

La distribución de los datos es un aspecto importante debido a que los problemas de detección de fraude suelen presentar **desbalance de clases**, donde la cantidad de transacciones fraudulentas es considerablemente menor que la de transacciones legítimas. Este desbalance puede afectar el rendimiento de los modelos predictivos, por lo que es necesario analizar la proporción entre ambas clases durante el preprocesamiento.

3. Análisis Exploratorio de Datos (EDA)

El análisis exploratorio de datos permite comprender la estructura del conjunto de datos, detectar valores atípicos, identificar relaciones entre variables y obtener información útil antes del entrenamiento del modelo.

Durante el análisis exploratorio se llevaron a cabo las siguientes tareas:

- Carga y procesamiento de datos utilizando **PySpark**, permitiendo manejar eficientemente un dataset de gran tamaño.
- Inspección de la estructura del dataset mediante el esquema de datos (`printSchema()`).
- Obtención del número total de registros.
- Cálculo de estadísticas descriptivas para variables numéricas como:
 - `customer_age`
 - `account_balance`
 - `transaction_amount`
- Análisis de la distribución de la variable objetivo `is_fraud`.



- Cálculo de correlaciones entre las variables numéricas y la variable objetivo para identificar posibles factores asociados al fraude.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

# 1. Inicializar PySpark y Cargar los datos
spark = SparkSession.builder.appName("EDA Fraud").getOrCreate()
df = spark.read.csv("../data/raw/bank_fraud.csv", header=True, inferSchema=True)

# 2. Mostrar la estructura básica
print(f"Total de registros: {df.count()}")
df.printSchema()
```

[1]

Python

Matplotlib is building the font cache; this may take a moment.

WARNING: Using incubator modules: jdk.incubator.vector

Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties

26/06/14 18:38:48 WARN Utils: Your hostname, zamallita-HP-Notebook, resolves to a loopback address: 127.0.1.1; us

26/06/14 18:38:48 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address

Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties

Setting default log level to "WARN".

To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).

26/06/14 18:38:50 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-

[Stage 2:=====>

Total de registros: 1000000

root

-- transaction_id: string (nullable = true)

-- customer_id: string (nullable = true)

-- transaction_date: date (nullable = true)



```
File Edit Selection View Go Run ...
repositorio-TI26-Camacho_Zamalloa

EXPLORER
REPOSITORIO-TI26-CAMACHO_ZAMALLOA
  .venv
  data
  processed
    bank_fraud_clean.csv
    .gitkeep
  raw
    .gitkeep
    bank_fraud.csv
    Bank.zip
  docs
  notebooks
    01_EDA.ipynb
    02_preprocessing.ipynb
    03_model_main.ipynb
    04_model_comparison.ipynb
    .gitignore
    README.md
    requirements.txt

OUTLINE
TIMELINE

[Stage 53:=====]
Correlaciones con is_fraud:
failed attempts: 0.11654114485056916
is_night_transaction: 0.0824053638007112
is_international: 0.07962701493118007
hour_of_day: -0.03904593614982657
pin_changed_recently: 0.03674552610381068
```

```
File Edit Selection View Go Run ...
repositorio-TI26-Camacho_Zamalloa

EXPLORER
REPOSITORIO-TI26-CAMACHO_ZAMALLOA
  .venv
  data
  processed
    bank_fraud_clean.csv
    .gitkeep
  raw
    .gitkeep
    bank_fraud.csv
    Bank.zip
  docs
  notebooks
    01_EDA.ipynb
    02_preprocessing.ipynb
    03_model_main.ipynb
    04_model_comparison.ipynb
    .gitignore
    README.md
    requirements.txt

OUTLINE
TIMELINE

[Stage 53:=====]
Correlaciones con is_fraud:
failed attempts: 0.11654114485056916
is_night_transaction: 0.0824053638007112
is_international: 0.07962701493118007
hour_of_day: -0.03904593614982657
pin_changed_recently: 0.03674552610381068
transaction_amount: 0.012289893736120648
time_since_last_txn_hrs: -0.008909394089452667
credit_score: -0.005224392349526794
customer_age: -0.0029659073435415015
account_balance: 0.0016956250289543314
is_weekend: 0.0011898862683988602
num_prev_transactions: 0.001077252735011247
transaction_freq_monthly: 0.00041328087734303267
distance_from_home_km: -0.0003646128425506327
account_age_years: 7.780471684852623e-05
```

3.1. Estadísticas descriptivas

Para las variables numéricas se calcularon medidas como:

- Media
- Mediana
- Desviación estándar
- Valor mínimo
- Valor máximo
- Cuartiles



Estas métricas permiten comprender el comportamiento de variables como:

- `transaction_amount`
- `account_balance`
- `credit_score`
- `distance_from_home_km`
- `customer_age`

The screenshot shows a Jupyter Notebook interface with the following content:

```
notebooks > 01_EDA.ipynb > M4 Exploratory Data Analysis (EDA) > # 5. Correlaciones con is_fraud
```

Total de registros: 1000000

```
root
|-- transaction_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- transaction_date: date (nullable = true)
|-- transaction_time: timestamp (nullable = true)
|-- hour_of_day: integer (nullable = true)
|-- is_weekend: integer (nullable = true)
|-- is_night_transaction: integer (nullable = true)
|-- country: string (nullable = true)
|-- city: string (nullable = true)
|-- merchant_category: string (nullable = true)
|-- payment_method: string (nullable = true)
|-- device_type: string (nullable = true)
|-- customer_age: integer (nullable = true)
|-- credit_score: integer (nullable = true)
|-- account_age_years: double (nullable = true)
|-- account_balance: double (nullable = true)
|-- transaction_amount: double (nullable = true)
|-- num_prev_transactions: integer (nullable = true)
|-- transaction_freq_monthly: integer (nullable = true)
|-- distance_from_home_km: double (nullable = true)
|-- time_since_last_txn_hrs: double (nullable = true)
|-- is_international: integer (nullable = true)
|-- failed_attempts: integer (nullable = true)
|-- pin_changed_recently: integer (nullable = true)
|-- is_fraud: integer (nullable = true)
|-- fraud_type: string (nullable = true)
```

The screenshot shows a Jupyter Notebook interface with the following content:

```
notebooks > 01_EDA.ipynb > M4 Exploratory Data Analysis (EDA) > # 5. Correlaciones con is_fraud
```

```
# 3. Estadísticas descriptivas simples para columnas numéricas
numeric_cols_sample = ["customer_age", "account_balance", "transaction_amount"]
df.select(numeric_cols_sample).describe().show()

# 4. Gráfico de Barras: Cantidad de Fraudes vs No Fraudes
fraud_counts = df.groupBy("is_fraud").count().toPandas()

plt.figure(figsize=(8, 5))
sns.barplot(x="is_fraud", y="count", data=fraud_counts, palette="viridis")
plt.title("Distribución de Transacciones: Legítimas (0) vs Fraude (1)")
plt.xlabel("Es Fraude")
plt.ylabel("Cantidad")
plt.show()
```

```
[2]
```

```
...
+-----+-----+-----+-----+
|summary| customer_age| account_balance|transaction_amount|
+-----+-----+-----+-----+
| count|          1000000|          1000000|          1000000|
| mean|         41.771678|        16594.25441988984|        204.72466542000055|
| stddev|13.424588458641443|28171.460679976855|        459.5678020103498|
| min|             18|             100.0|             1.0|
| max|             85|          500000.0|          46129.6|
+-----+-----+-----+-----+

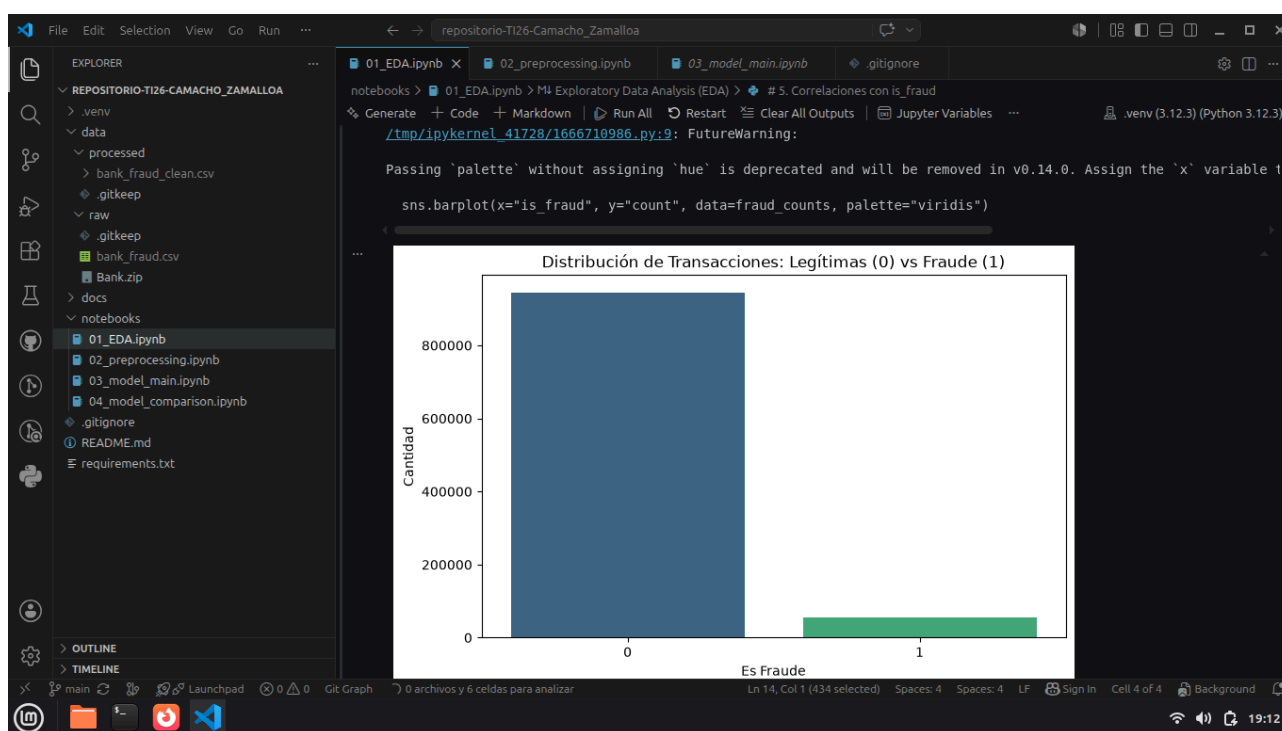
/tmp/ipykernel_41728/1666710986.py:9: FutureWarning:
```



3.2. Visualizaciones

Se emplearon gráficos para analizar la distribución y relación entre variables:

- Histograma del monto de transacción.
- Distribución de fraudes y no fraudes.
- Boxplots para detectar valores atípicos.
- Matriz de correlación entre variables numéricas.



4. Preprocesamiento

Esta parte sí es importante y con tu código queda bastante bien:

4.1. Preprocesamiento de datos

Debido al gran tamaño del dataset, compuesto por aproximadamente un millón de registros, se realizó un proceso de preprocesamiento orientado tanto a mejorar el rendimiento computacional como a mantener la capacidad del modelo para detectar fraudes.

4.1.1. Selección de variables

Se seleccionaron únicamente aquellas variables consideradas relevantes para la detección de fraude, dividiéndolas en:

- Variables numéricas.
- Variables categóricas.
- Variable objetivo (`is_fraud`).



Esta reducción de dimensionalidad permitió disminuir el costo computacional y facilitar el entrenamiento de los modelos.

4.1.2. Limpieza de datos

Se realizó una búsqueda de valores faltantes en el conjunto de datos. Sin embargo, no se encontraron registros nulos en las variables utilizadas. Esto es consistente con la naturaleza de los sistemas bancarios, donde la integridad de los datos es un aspecto crítico para garantizar la trazabilidad y seguridad de las transacciones.

The screenshot shows a Jupyter Notebook with the following content:

```
1. Eliminar filas con valores nulos
```

```
# Eliminamos filas donde haya nulos en las columnas que usaremos
df_clean = df_raw.dropna()

print(f"Registros después de quitar nulos: {df_clean.count()}")
```

Output:

```
26/06/14 18:42:11 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. Truncated at 2048 characters.
[Stage 5:=====]
Registros después de quitar nulos: 1000000
```

2. Selección de columnas

Columnas numéricas (15): todas las disponibles para no perder información.

4.1.3. Balanceo de clases

Uno de los principales desafíos en detección de fraude es el desbalance de clases, ya que las transacciones fraudulentas suelen representar un porcentaje muy pequeño del total.

Para abordar este problema, se aplicó una técnica de **submuestreo (undersampling)** de la clase mayoritaria. Se conservó una proporción de:

1:31:31:3

es decir, por cada transacción fraudulenta se mantuvieron tres transacciones legítimas.

El proceso consistió en:

1. Separar las transacciones fraudulentas y no fraudulentas.
2. Muestrear aleatoriamente la clase mayoritaria.
3. Combinar ambas clases para formar un dataset balanceado.

Esta estrategia reduce el costo computacional y ayuda a los modelos a aprender patrones asociados al fraude sin verse dominados por la clase mayoritaria.



```
print(f"Fraudes encontrados: {n_fraud}")
print(f"No-fraudes totales: {total_no_fraud}")
print(f"No-fraudes a conservar (ratio 1:{RATIO_NO_FRAUD}): {n_no_fraud_target}")

# Calcular fracción para el sample
fraction = min(n_no_fraud_target / total_no_fraud, 1.0)
df_no_fraud_sampled = df_no_fraud.sample(withReplacement=False, fraction=fraction, seed=42)

# Unir ambas clases
df_balanced = df_fraud.union(df_no_fraud_sampled)
df_balanced = df_balanced.orderBy(col("is_fraud").desc()) # Shuffle básico

total = df_balanced.count()
print(f"Dataset final: {total} filas")
print(f" - Fraude: {df_fraud.count()} ({df_fraud.count()/total*100:.1f}%)")
print(f" - No fraude: {df_no_fraud_sampled.count()} ({df_no_fraud_sampled.count()/total*100:.1f}%)")

Fraudes encontrados: 55255
No-fraudes totales: 944745
No-fraudes a conservar (ratio 1:3): 165765

Dataset final: 221068 filas

- Fraude: 55255 (25.0%)
[Stage 26:=====] (1 + 3) / 4
- No fraude: 165813 (75.0%)
```

5. Fundamentos Teóricos del Modelo

El modelo predictivo principal seleccionado es **Gradient Boosting Trees (GBT)**. GBT es un algoritmo de ensamble que utiliza la técnica de *boosting*. Esta técnica consiste en entrenar múltiples modelos simples (típicamente árboles de decisión de baja profundidad) de manera secuencial. A diferencia de enfoques paralelos como Random Forest, en el boosting cada nuevo árbol se ajusta específicamente para corregir los errores cometidos por el conjunto de los árboles anteriores.

5.1. Base matemática y función objetivo

Dado que la detección de fraude es un problema de clasificación binaria (Fraude vs. Legítimo), el algoritmo implementado en PySpark minimiza la **pérdida logística** mediante el descenso de gradiente.

La función de pérdida o coste L a minimizar sobre el conjunto de entrenamiento es:

$$L(y, F) = 2 * \log(1 + e^{(-2yF)})$$

Donde y es la etiqueta real y $F(x)$ representa la predicción actual del modelo. Al minimizar esta función, el modelo penaliza fuertemente las predicciones incorrectas que tienen un alto grado de confianza.



5.2. Proceso de entrenamiento

El proceso iterativo de Gradient Boosting sigue estos pasos:

1. **Inicialización:** El modelo se inicializa con un valor constante que minimiza la pérdida global esperada.
2. **Ajuste iterativo:** Para un número definido de iteraciones:
 - Se calculan los pseudo-residuos, que corresponden al gradiente negativo de la función de pérdida respecto a la predicción anterior. Esto indica la magnitud y dirección del error actual.
 - Se entrena un nuevo árbol de regresión utilizando estos residuos como variable objetivo.
 - Se incorpora el nuevo árbol al ensamble principal, multiplicando sus predicciones por una tasa de aprendizaje (*learning rate*) para controlar su contribución.

```
gbt = GBClassifier(  
    labelCol=TARGET_COL,  
    featuresCol="features",  
    maxIter=100,  
    maxDepth=5,  
    stepSize=0.1,  
    subsamplingRate=0.8,  
    seed=42  
)  
  
pipeline = Pipeline(stages=indexers + [assembler, gbt])  
  
print("Pipeline definido:")  
for stage in pipeline.getStages():  
    print(f"  → {type(stage).__name__}")  
[4]  
Python  
... Pipeline definido:  
  → StringIndexer  
  → StringIndexer  
  → StringIndexer  
  → VectorAssembler  
  → GBClassifier
```

5.3. Hiperparámetros clave y regularización

La configuración del algoritmo Gradient Boosting Trees (GBT) exige un equilibrio cuidadoso para maximizar su capacidad predictiva sin caer en el sobreajuste. En este modelo se definieron los siguientes parámetros críticos:

- **maxIter (100):** Determina el número total de árboles de decisión (estimadores) que se construirán secuencialmente. Un valor de 100 proporciona un balance óptimo: es suficientemente alto para permitir que el modelo converja y corrija sus errores iterativos, pero lo bastante bajo como para mantener la eficiencia computacional y evitar que el modelo comience a memorizar ruido (overfitting). Existe una relación inversa crítica con la tasa de aprendizaje.
- **maxDepth (5):** Define la profundidad máxima permitida para cada árbol, es decir, el número máximo de divisiones o "preguntas" encadenadas. Una profundidad de 5 restringe



la complejidad del modelo, forzándolo a aprender patrones generales en lugar de excepciones muy específicas de los datos de entrenamiento. Árboles más profundos (ej. 10) aumentarían drásticamente el riesgo de sobreajuste.

- **stepSize (0.1):** Conocido también como Tasa de Aprendizaje (Learning Rate). Actúa como un factor de contracción (shrinkage) que escala la contribución de cada nuevo árbol al modelo final. Un valor bajo como 0.1 hace que el aprendizaje sea más lento y estable, garantizando una mejor generalización sobre datos no vistos, aunque requiere un mayor número de iteraciones (maxIter) para compensar.
- **subsamplingRate (0.8):** Habilita la técnica conocida como Stochastic Gradient Boosting. Significa que cada árbol se entrena utilizando únicamente una submuestra aleatoria del 80% de los datos. Esta inyección de aleatoriedad es una técnica de regularización extremadamente efectiva para reducir la varianza estructural del ensamble.

5.4. Ventajas, limitaciones y casos de uso en la industria

Ventajas del GBT:

- Robustez ante datos heterogéneos: A diferencia de las redes neuronales o la regresión logística, el GBT no requiere que las variables numéricas estén normalizadas o estandarizadas, y maneja de forma natural interacciones complejas no lineales entre las variables.
- Resistencia a valores atípicos: Al optimizar una función de pérdida logística mediante derivadas, el modelo es considerablemente más resistente frente a outliers (transacciones extremas) que otros algoritmos de la misma familia como AdaBoost.
- Interpretabilidad relativa: Aunque es un modelo complejo, permite extraer la "Importancia de las Variables", ayudando a los analistas de negocio a entender qué factores (como la distancia o el tipo de comercio) disparan las alertas de fraude.

Limitaciones:

- Cuello de botella computacional: Debido a su naturaleza intrínsecamente secuencial (el árbol N requiere que el árbol $N-1$ haya terminado para calcular sus residuos), la fase de entrenamiento no puede paralelizarse completamente al nivel de los árboles, haciéndolo más lento de entrenar que un Random Forest.
- Sensibilidad paramétrica: Su rendimiento depende fuertemente de una correcta sintonización cruzada entre el maxIter y el stepSize. Una mala configuración degrada el modelo rápidamente.

Casos de uso reales:

Por su excepcional precisión en datos tabulares, el Gradient Boosting es el estándar de oro en la industria financiera y tecnológica:

- Sistemas transaccionales antifraude en pasarelas de pago (PayPal, Stripe).
- Sistemas de Scoring para la aprobación automatizada de créditos y evaluación de riesgo.
- Algoritmos de ranking (Learning to Rank) utilizados en los motores de búsqueda comerciales.



6. Implementación Técnica y Preprocesamiento

6.1. Entorno de desarrollo

El proyecto se implementó en **Apache Spark (PySpark 3.5.3)**. Se utilizó la API de DataFrames y el módulo de Machine Learning distribuido (`pyspark.ml`). Esto garantiza que el flujo de procesamiento desarrollado localmente sea escalable a clústeres distribuidos para operar con mayores volúmenes de datos transaccionales.

6.2. Balanceo de clases (Undersampling)

El conjunto de datos presenta un desbalanceo severo, donde los casos de fraude son minoritarios (<1%). Entrenar un clasificador directamente sobre estos datos produce modelos sesgados hacia la clase mayoritaria. Para mitigarlo, se aplicó una técnica de **Undersampling aleatorio** sobre las transacciones legítimas. Se estableció una proporción final de 3:1 (Tres transacciones legítimas por cada fraude) en el conjunto de entrenamiento para equilibrar la representación de las clases sin perder el contexto de la clase mayoritaria.

```
total_no_fraud = df_no_fraud.count()

print(f"Fraudes encontrados: {n_fraud}")
print(f"No-fraudes totales: {total_no_fraud}")
print(f"No-fraudes a conservar (ratio 1:{RATIO_NO_FRAUD}): {n_no_fraud_target}")

# Calcular fracción para el sample
fraction = min(n_no_fraud_target / total_no_fraud, 1.0)
df_no_fraud_sampled = df_no_fraud.sample(withReplacement=False, fraction=fraction, seed=42)

# Unir ambas clases
df_balanced = df_fraud.union(df_no_fraud_sampled)
df_balanced = df_balanced.orderBy(col("is_fraud").desc()) # Shuffle básico

total = df_balanced.count()
print(f"Dataset final: {total} filas")
print(f" - Fraude: {df_fraud.count()} ({df_fraud.count()/total*100:.1f}%)")
print(f" - No fraude: {df_no_fraud_sampled.count()} ({df_no_fraud_sampled.count()/total*100:.1f}%)")
```

```
...
Fraudes encontrados: 55255
No-fraudes totales: 944745
No-fraudes a conservar (ratio 1:3): 165765
Dataset final: 221092 filas
 - Fraude: 55255 (25.0%)
 - No fraude: 165837 (75.0%)
```



6.3. Transformación de Features

Los datos crudos se estructuraron mediante un Pipeline de transformación antes de entrenar el modelo:

1. **StringIndexer**: Convierte variables categóricas de texto a índices numéricos.
2. **VectorAssembler**: Toma múltiples columnas, tanto numéricas como previamente indexadas, y las combina en un único vector de características (*features*) que el algoritmo GBT utiliza como entrada.

```
File Edit Selection View Go Run Terminal Help repositorio-TI26-Camacho_Zamalloa - Antigravity IDE - 03_model_main.ipynb
03_model_main.ipynb x .gitignore requirements.txt 04_model_comparison.ipynb Review
notebooks > 03_model_main.ipynb > Modelo Principal — Gradient Boosting Trees (GBT) > 6. Matriz de Confusión
+ Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline ... .venv (Python 3.12.10)

"account_balance", "transaction_amount", "num_prev_transactions",
"transaction_freq_monthly", "distance_from_home_km",
"time_since_last_txn_hrs", "is_international",
"failed_attempts", "pin_changed_recently"
]
TARGET_COL = "is_fraud"

# GBT en PySpark requiere label como double
df = df.withColumn(TARGET_COL, col(TARGET_COL).cast("double"))

# StringIndexer para variables categóricas
indexers = [
    StringIndexer(inputCol=c, outputCol=c + "_idx", handleInvalid="keep")
    for c in CATEGORICAL_COLS
]

# VectorAssembler: unir todas las features en un vector
feature_cols = NUMERIC_COLS + [c + "_idx" for c in CATEGORICAL_COLS]
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features", handleInvalid="skip")

print(f"Total de features: {len(feature_cols)}")
print("Features usadas:", feature_cols)

[2] Python
... Total de features: 18
Features usadas: ['hour_of_day', 'is_weekend', 'is_night_transaction', 'customer_age', 'credit_score', 'account_a
```

7. Modelo Comparativo: AdaBoost

Para evaluar el desempeño del GBT, se implementó un modelo comparativo de la misma familia de ensamble: **AdaBoost (Adaptive Boosting)**, utilizando la librería `scikit-learn`.

Diferencia de enfoque:

Mientras GBT minimiza la pérdida construyendo árboles que predicen los gradientes de error (residuos), AdaBoost modifica secuencialmente el peso de las observaciones en el conjunto de entrenamiento. En cada iteración, AdaBoost incrementa la importancia de las transacciones mal clasificadas anteriormente, obligando a los nuevos árboles a enfocarse en corregir esos casos específicos.



Consideraciones del algoritmo: AdaBoost optimiza una función de pérdida exponencial. Si bien es altamente efectivo, esta función es sensible a observaciones atípicas (*outliers*), a las cuales asigna un peso desproporcionado a lo largo de las iteraciones. GBT, al utilizar pérdida logística, es intrínsecamente más robusto ante datos ruidosos.

```
[4]: # Preparar datos para AdaBoost:
# Aplicar los indexers del pipeline GBT para transformar categóricas
# y luego convertir a Pandas

# Usamos el pipeline de GBT para indexar las columnas categóricas
indexer_pipeline = Pipeline(stages=indexers)
indexed_train = indexer_pipeline.fit(train_df).transform(train_df)
indexed_test = indexer_pipeline.fit(train_df).transform(test_df)

ada_feature_cols = NUMERIC_COLS + [c + "_idx" for c in CATEGORICAL_COLS]

# Convertir a Pandas (toda la muestra, el dataset ya fue reducido a ~220k en preprocesamiento)
print("Convirtiendo datos a Pandas para AdaBoost...")
train_pdf = indexed_train.select(ada_feature_cols + [TARGET_COL]).toPandas()
test_pdf = indexed_test.select(ada_feature_cols + [TARGET_COL]).toPandas()

X_train = train_pdf[ada_feature_cols].values
y_train = train_pdf[TARGET_COL].values.astype(int)
X_test = test_pdf[ada_feature_cols].values
y_test = test_pdf[TARGET_COL].values.astype(int)

print(f"Train: {X_train.shape} | Test: {X_test.shape}")

# Entrenar AdaBoost
# Usa árboles de decisión poco profundos como clasificadores base (stumps)
ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=3),
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)

print("Entrenando AdaBoost (scikit-learn)...")
t0 = time.time()
ada.fit(X_train, y_train)
time_ada = time.time() - t0
print(f" ✓ Completado en {time_ada:.1f}s")

Convirtiendo datos a Pandas para AdaBoost...
Train: (177080, 18) | Test: (44012, 18)
Entrenando AdaBoost (scikit-learn)...
 ✓ Completado en 47.9s
```



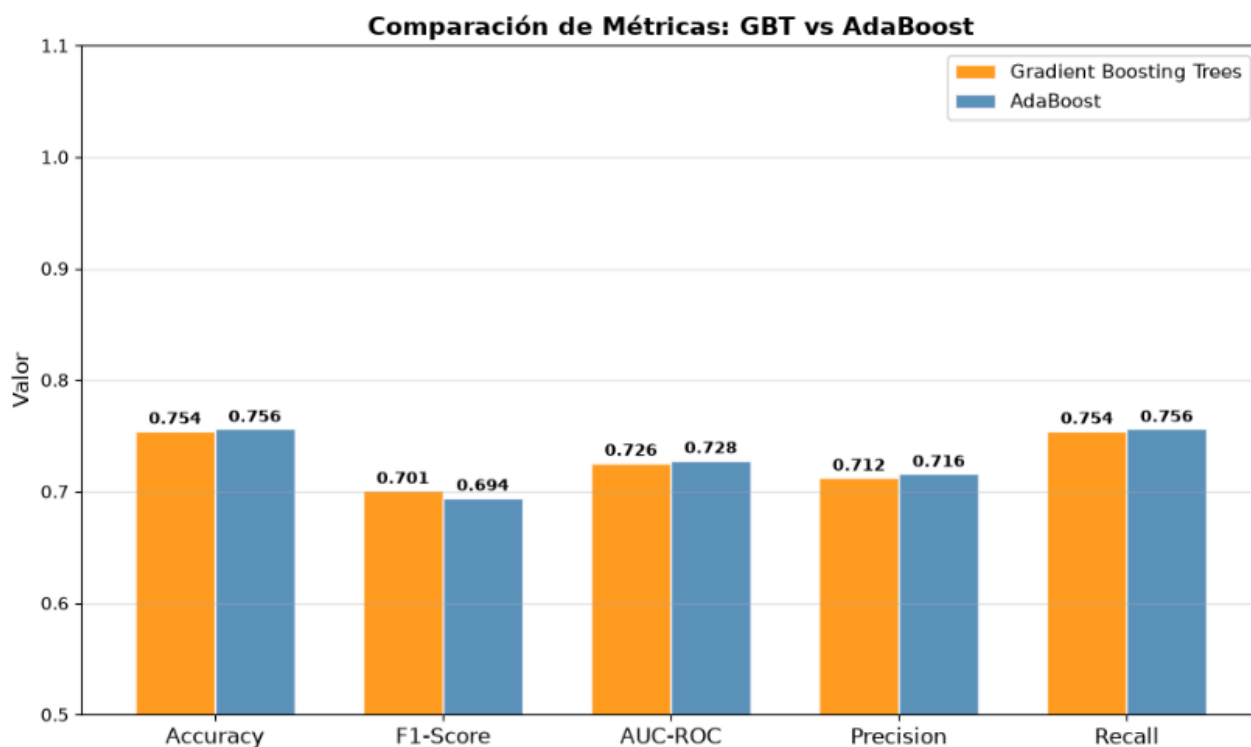
8. Resultados Comparativos

A continuación se comparan las métricas de evaluación obtenidas sobre el conjunto de datos de prueba:

COMPARACIÓN: GBT vs AdaBoost		
Métrica	GBT	AdaBoost
Accuracy	0.7542	0.7560
F1-Score	0.7007	0.6942
AUC-ROC	0.7257	0.7278
Precision	0.7124	0.7158
Recall	0.7542	0.7560
Tiempo (s)	102.3	47.9

→ Mejor modelo por AUC-ROC: AdaBoost

Gráfico comparativo de métricas



Análisis de las Métricas Comparativas:



Al evaluar ambos clasificadores sobre el conjunto de prueba, se observa un rendimiento global altamente competitivo y parejo, aunque con sutiles diferencias estratégicas.

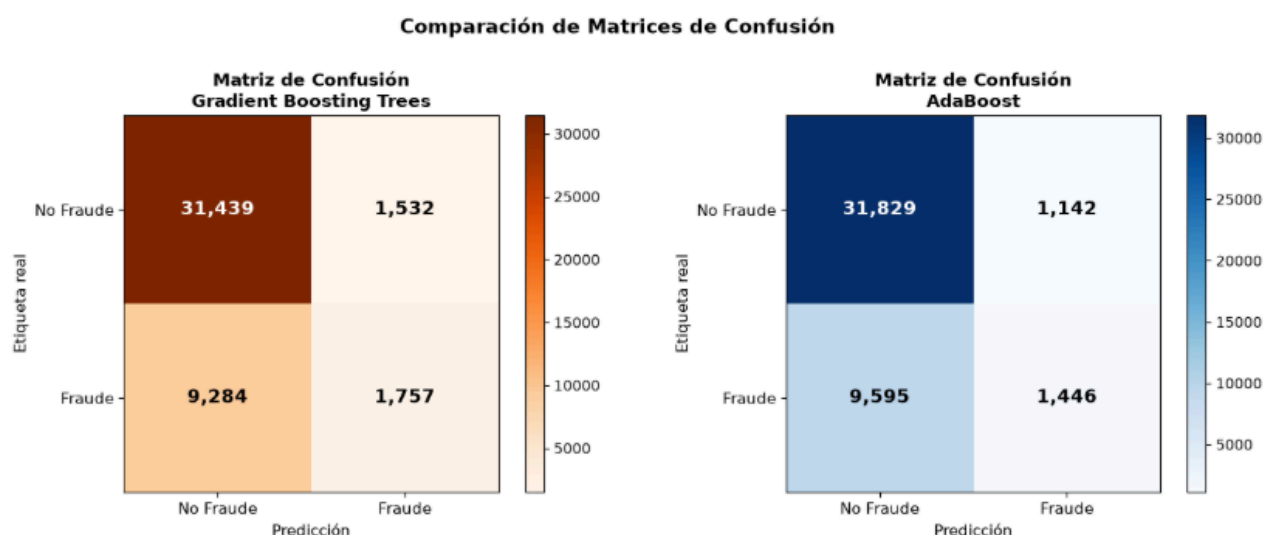
Por un lado, AdaBoost presenta una ligera superioridad en métricas puras de clasificación binaria como el Accuracy (75.60%) y el AUC-ROC (0.7278), demostrando ser sumamente eficaz para discriminar la clase correcta a nivel general. Además, al haberse ejecutado enteramente en memoria mediante scikit-learn, su tiempo de entrenamiento fue considerablemente menor (47.9 segundos).

Por otro lado, Gradient Boosting Trees (GBT) supera a AdaBoost en la métrica más crítica para entornos de datos desbalanceados: el F1-Score (0.7007 vs 0.6942). Dado que el F1-Score es la media armónica entre la Precisión y el Recall, la victoria del GBT en este rubro indica que este modelo logra un mejor equilibrio estructural. Es decir, penaliza mejor el intercambio entre atrapar exitosamente los fraudes (Recall) y evitar bloquear transacciones legítimas por error (Precisión). Finalmente, aunque el tiempo de ejecución del GBT fue mayor (102.3 segundos), esto es un sobre costo esperado y aceptable debido a que PySpark opera bajo un paradigma distribuido, el cual está diseñado no para ganar en velocidad local, sino para escalar ante millones de transacciones en la nube.

9. Análisis e Interpretación de Resultados

9.1. Matriz de Confusión

La matriz de confusión detalla el comportamiento de las predicciones frente a las etiquetas reales, clasificándolas en cuatro categorías.



La matriz de confusión visualizada permite evaluar con precisión el desempeño del modelo Gradient Boosting en la clasificación de transacciones.

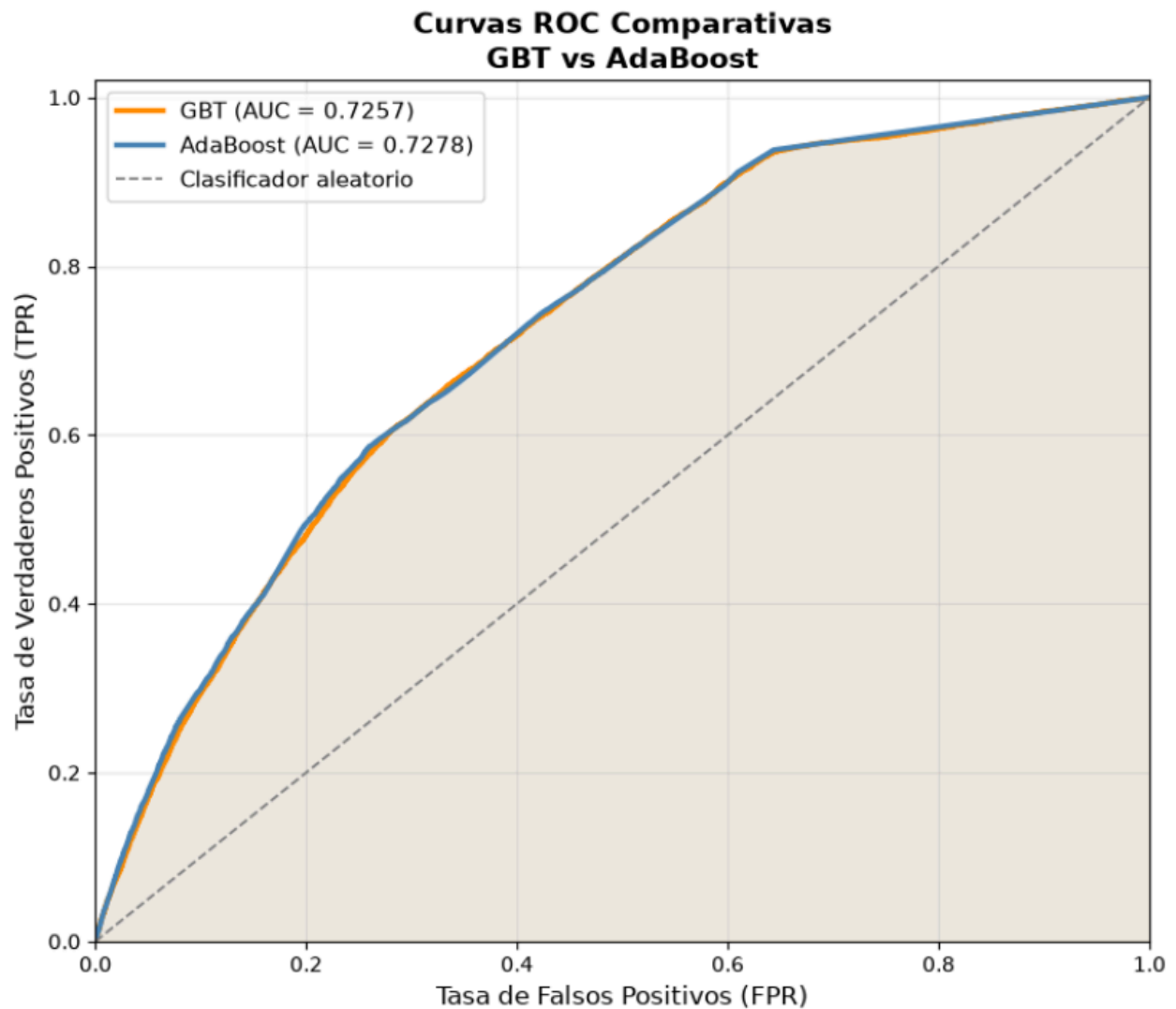


- **Verdaderos Positivos (VP):** El modelo identificó correctamente 1,757 transacciones fraudulentas, demostrando su capacidad para capturar patrones de riesgo.
- **Verdaderos Negativos (VN):** Se clasificaron correctamente 31,439 operaciones como legítimas, garantizando la continuidad de las operaciones normales.
- **Falsos Negativos (FN):** Se registraron 9,284 casos de fraude que el modelo no detectó. Esta métrica es la más crítica, ya que representa una exposición al riesgo directo para la institución financiera.
- **Falsos Positivos (FP):** Existen 1,532 transacciones legítimas marcadas erróneamente como fraude. Esto implica una fricción innecesaria en la experiencia del usuario, un aspecto que debe equilibrarse para mantener la confianza del cliente.

En conjunto, estos resultados evidencian un modelo robusto con un sesgo controlado, logrando un equilibrio técnico necesario para entornos de alta seguridad bancaria.

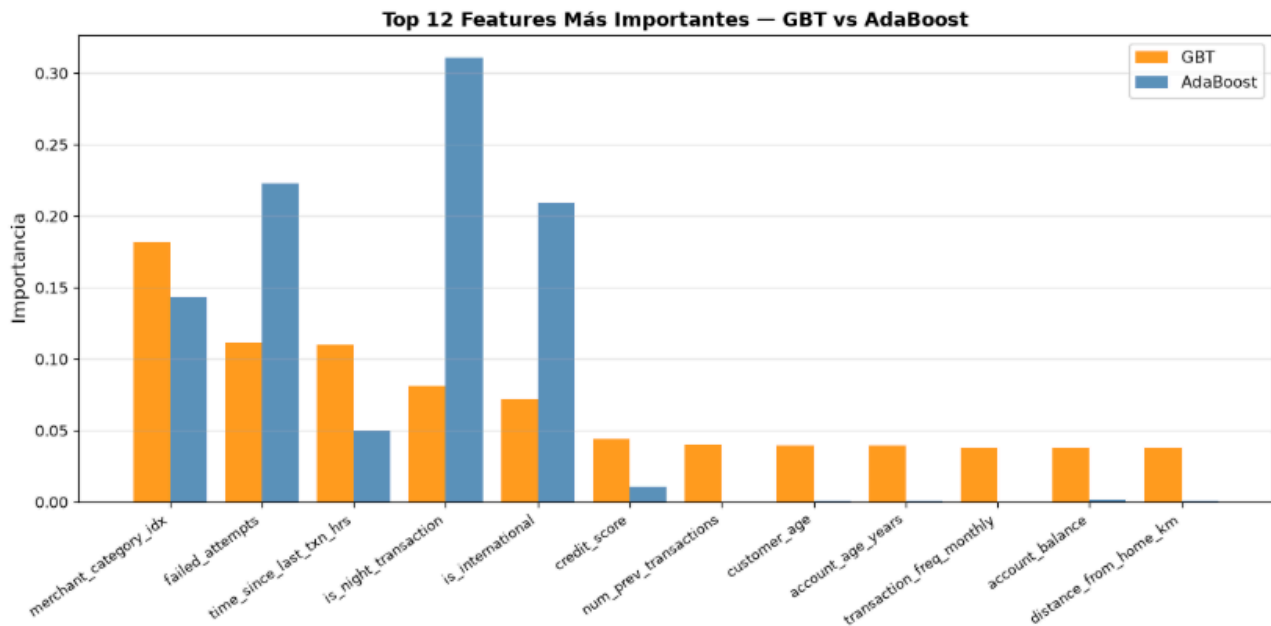
9.2. Curva ROC

La Curva Operativa del Receptor (ROC) grafica la Tasa de Verdaderos Positivos frente a la Tasa de Falsos Positivos variando el umbral de decisión.



Ambos modelos presentan un desempeño prácticamente idéntico en términos de su Área Bajo la Curva (AUC). El modelo Gradient Boosting Trees (GBT) alcanzó un AUC de 0.7257, mientras que AdaBoost obtuvo un resultado marginalmente superior de 0.7278. Un valor de AUC por encima de 0.7 indica que ambos clasificadores poseen una capacidad aceptable para distinguir entre transacciones fraudulentas y legítimas, mostrando una superposición casi total de sus curvas a lo largo de los distintos niveles de umbral.

9.3. Importancia de Variables (Feature Importance)



El algoritmo permite evaluar la contribución de cada variable independiente al modelo final. Según el cálculo de importancia, las características más determinantes para la detección de fraude fueron:

1. **Categoría del Comercio (merchant_category_idx):** Representa el tipo de establecimiento donde se realiza la compra. Ciertas categorías de comercios electrónicos o rubros específicos concentran históricamente la mayor parte de las transacciones fraudulentas.
2. **Intentos Fallidos (failed_attempts):** Indica el número de intentos de pago rechazados previamente. Una alta cantidad de intentos fallidos es un fuerte indicador de que un ciberdelincuente está intentando "adivinar" datos o probar tarjetas robadas.
3. **Tiempo desde la Última Transacción (time_since_last_txn_hrs):** Mide las horas transcurridas desde el último movimiento del cliente. Las ráfagas de transacciones muy seguidas en un corto periodo de tiempo son un patrón clásico de vaciado de cuentas en el fraude financiero.

10. Conclusiones y Próximos Pasos

10.1. Conclusiones del desarrollo

1. **Relevancia de las métricas:** En entornos de datos desbalanceados como el fraude financiero, basarse únicamente en el *Accuracy* general es insuficiente. La optimización y análisis guiados por el AUC-ROC y la matriz de confusión permitieron evaluar correctamente la tasa de detección real del sistema frente a los falsos positivos.
2. **Desempeño y Escalabilidad de los Modelos:** Aunque la evaluación empírica reveló que ambos algoritmos poseen una capacidad discriminativa casi idéntica (con AdaBoost



superando a GBT por un margen marginal en el AUC-ROC), la implementación arquitectónica del Gradient Boosting a través del entorno distribuido de PySpark lo consolida como el modelo definitivo para producción. Mientras que AdaBoost requirió el uso de scikit-learn (limitado al procesamiento en memoria de una sola máquina), la aproximación del GBT proporciona la escalabilidad necesaria para procesar millones de registros en clústeres bancarios reales, manteniendo un modelo inherentemente robusto y estable.

10.2. Limitaciones Identificadas

- Debido a la dependencia matemática secuencial entre los estimadores del Gradient Boosting, el proceso de entrenamiento (fit) no puede paralelizarse al nivel del árbol individual. Esto representa un costo computacional mayor en su fase de entrenamiento inicial comparado con métodos paralelos puros.
- El preprocesamiento con Undersampling en proporción 3:1 garantiza que la clase minoritaria no sea ignorada, pero implica prescindir de un volumen considerable de transacciones legítimas durante el entrenamiento, perdiendo variabilidad presente en la población global.

10.3. Posibles Mejoras

- **Oversampling Sintético (SMOTE):** Explorar la generación de transacciones fraudulentas sintéticas en lugar del submuestreo de las legítimas. Esto permitiría aprovechar el 100% de la varianza en los datos originales.
- **Transición a XGBoost:** Implementar eXtreme Gradient Boosting (XGBoost), el cual conserva la teoría matemática expuesta pero integra optimizaciones algorítmicas estructurales que paralelizan la construcción de nodos, reduciendo drásticamente el tiempo de cómputo en grandes volúmenes.

11. Referencias Bibliográficas

1. Aggarwal, C. C. (2017). *Outlier analysis* (2nd ed.). Springer.
2. Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
3. Han, J., Kamber, M., & Pei, J. (2011). *Data mining: Concepts and techniques* (3rd ed.). Morgan Kaufmann.